
535510 RL Team Project: GLIDE: GRU-based Lag-Informed Decision Engine for GPU Job Dispatching

Haopei Teng Juiyun Chang Jinjia Zheng

Department of Computer Science

National Yang Ming Chiao Tung University

{jxiiijll.cs14, ray314551145.cs14, howgiyo.cs14}@nycu.edu.tw

Abstract and project goal: Modern GPU clusters must balance multiple competing objectives, including latency, slowdown, fairness, and resource fragmentation. Existing GPU cluster schedulers often rely on heuristic or rule-based scheduling policies, which remain strong and practical baselines in real cluster environments [Gu et al., 2019, Xiao et al., 2018, Qiao et al., 2021, Mahajan et al., 2020]. However, fixed dispatching rules such as FIFO, SJF, slowdown priority, and resource-aware heuristics are limited in their ability to adapt decisions to the current combination of queue pressure, resource availability, and accumulated service imbalance.

This project studies whether reinforcement learning can improve the *dispatching order* of GPU jobs under a fixed placement backend. It proposes **GLIDE**, a service-lag-aware GRU actor-critic scheduler trained with PPO through online interaction with a trace-driven GPU cluster simulator. GLIDE observes a bounded visible queue window, filters jobs that are currently infeasible under the placement backend, and selects one feasible job to dispatch, with an optional pass action.

Instead of claiming formal guarantees on fairness or SLA compliance, GLIDE optimizes a penalty-based composite score and uses a CFS-style normalized service-lag metric as the primary checkpoint-selection signal. This design is motivated by fairness-aware scheduling and constrained reinforcement learning, where a policy should improve the target objective without causing unacceptable degradation in other system metrics [Wong et al., 2008, Achiam et al., 2017, Altman, 2021]. The goal is to reduce persistent under-service among GPU-demand groups while preserving conventional scheduling metrics such as average waiting time, tail waiting time, slowdown, and Jain fairness [Jain et al., 1984].

We do not claim state-of-the-art GPU scheduling performance. Instead, this project focuses on a controlled dispatching comparison, where all evaluated methods share the same trace split, node subset, placement backend, and evaluation metrics.

TL;DR (“Too Long; Didn’t Read”): This project trains a PPO-based GRU dispatching policy, GLIDE, in a trace-driven GPU cluster simulator. GLIDE keeps First-Fit placement fixed and learns only which feasible job to launch from a bounded visible queue window, aiming to reduce CFS-style service-lag imbalance while keeping latency and slowdown within a configurable tolerance envelope.

1 Motivation

GPU resources are scarce, and modern deep learning and high-performance computing workloads are highly dynamic. Optimizing solely for utilization or short-term latency can lead to undesirable outcomes such as long-tail waiting time, unfair service across GPU-demand groups, and starvation-like behavior. Prior GPU cluster scheduling systems have shown that scheduling decisions strongly affect job completion time, slowdown, fairness, and resource efficiency [Gu et al., 2019, Xiao et al., 2018, Qiao et al., 2021, Mahajan et al., 2020]. As a result, GPU scheduling can be viewed as a se-

quential decision-making problem, where each dispatching action affects future queue composition, resource availability, and service balance.

Heuristic approaches such as FIFO, SJF, slowdown priority, best-fit-job, min-fragmentation, and max-utilization are simple and efficient. They are also difficult baselines because their rules often work well on average in practical scheduling environments. Nevertheless, these methods use fixed priorities and cannot explicitly adjust their dispatching behavior according to the current service-lag state. Reinforcement learning has been explored as a way to learn adaptive scheduling and resource-management policies from simulator interaction or workload traces [Mao et al., 2016, 2019, de Freitas Cunha and Chaimowicz, 2023, Yang et al., 2025]. However, learned policies must be carefully constrained in practice: a scheduler should not improve one metric by making waiting time, slowdown, or job completion reliability collapse [Achiam et al., 2017, Altman, 2021].

Therefore, there remains a need for scheduling methods that are adaptive, trace-evaluable, and able to improve service-lag fairness while remaining competitive with strong heuristic baselines.

Why is the problem interesting? GPU scheduling is a real-world, long-horizon decision-making problem in which actions have delayed and system-wide consequences. It involves structured queue states, dynamic resource availability, placement feasibility constraints, and multiple competing objectives. These properties make it a meaningful testbed for practical reinforcement learning beyond toy control environments, especially because prior RL-based schedulers have shown the potential of learning scheduling policies from complex cluster states [Mao et al., 2016, 2019, de Freitas Cunha and Chaimowicz, 2023], though adapting these learned policies efficiently across highly dynamic workload variations often necessitates advanced techniques like meta-learning [Yang et al., 2023].

Critical challenges:

Challenge 1: Strong heuristic baselines. Simple dispatching policies such as SJF and slowdown priority can already achieve strong latency and slowdown performance. A learned policy must therefore improve a meaningful metric, such as service-lag imbalance, without merely matching or degrading conventional scheduling quality.

Challenge 2: Fairness-efficiency trade-off. Reducing service imbalance is not sufficient if the scheduler achieves it by delaying jobs, increasing tail latency, or leaving jobs unfinished. The policy must improve service-lag behavior while staying within an acceptable efficiency envelope. This trade-off is consistent with fairness-aware GPU scheduling studies, where fairness improvements must be evaluated together with efficiency-related metrics [Mahajan et al., 2020, Qiao et al., 2021].

Challenge 3: Event-driven and partially observed dynamics. The scheduler observes only arrived jobs and current cluster resources, while future arrivals and completions are unknown. Moreover, decisions occur at irregular event times rather than fixed time steps, making the problem naturally event-driven and long-horizon. This motivates an SMDP-style formulation for cluster scheduling, where actions may have variable temporal durations and delayed effects [de Freitas Cunha and Chaimowicz, 2023].

Why is the problem still open? Existing heuristic schedulers are efficient and interpretable, but their fixed rules cannot adapt to service-lag dynamics. Learning-based schedulers can adapt to complex queue and resource states, but they are vulnerable to reward hacking and instability if model selection only follows a single scalar reward. In trace-driven GPU scheduling, it is also difficult to show clear gains over strong heuristics because many standard metrics, such as waiting time and slowdown, are highly correlated and are often already optimized well by simple rules [Feitelson, 2001, Gu et al., 2019, Mahajan et al., 2020]. A practical learned scheduler must therefore show improvements on a meaningful fairness-oriented metric while preserving conventional latency and slowdown metrics.

Relevant prior works:

- Classical scheduling heuristics such as FIFO, SJF, and slowdown-based priority rules provide strong and interpretable baselines for queue ordering [Feitelson, 2001, Gu et al., 2019].

- Resource-aware and fragmentation-aware scheduling heuristics select jobs based on placement feasibility, resource packing, or utilization, which are important concerns in GPU and deep learning cluster scheduling [Xiao et al., 2018, Narayanan et al., Qiao et al., 2021].
- Reinforcement learning methods for scheduling learn adaptive policies from simulator interaction or workload traces, but require careful reward design and validation to avoid degenerate scheduling behavior [Mao et al., 2016, 2019, de Freitas Cunha and Chaimowicz, 2023, Ryu and Eo, 2023, Yang et al., 2025].
- Fair-share scheduling ideas, including CFS-style service-lag tracking, motivate the primary service-imbalance metric used in this project [Wong et al., 2008, Jain et al., 1984, Mahajan et al., 2020].

State-of-the-art: Yang et al. [2025] is used as a recent representative learning-based scheduling method because it combines offline imitation learning with online adaptation in dynamic workflow scheduling. However, its main focus is workflow scheduling performance rather than explicit service-lag fairness or SLA-style efficiency gates. GLIDE therefore focuses on a different but related objective: improving service-lag-aware GPU job dispatching under a fixed placement backend while using constrained reinforcement learning ideas to discourage fairness gains that come at the cost of severe latency, slowdown, or completion degradation [Achiam et al., 2017, Altman, 2021].

Therefore, we treat these recent learning-based schedulers as representative related directions rather than directly reproduced baselines. Direct reproduction would introduce differences in simulator design, workload assumptions, placement mechanisms, and target objectives. To avoid such confounding factors, our empirical comparison keeps placement fixed and evaluates only the dispatching decision under the same trace-driven environment.

Expected research contributions. This project aims to make the following research contributions:

- We formulate service-lag-aware GPU job dispatching as an event-driven reinforcement learning problem under a fixed placement backend.
- We propose GLIDE, a GRU-based PPO dispatching policy with feasible-action filtering and service-lag-aware reward shaping.
- We design a controlled trace-driven evaluation protocol that compares learned dispatching against strong heuristic baselines under the same First-Fit placement backend.
- We use CFS normalized service-lag AUC together with efficiency-gated checkpoint selection to evaluate whether fairness-oriented improvements are achieved without severe latency or slowdown degradation.

2 Problem Formulation

GPU cluster scheduling is formulated as a trace-driven, event-driven reinforcement learning problem. Unlike fixed-step control tasks, scheduling decisions are triggered only when the system state changes, such as when jobs arrive, jobs complete, or resources become available. This event-driven structure is naturally modeled as a Semi-Markov Decision Process (SMDP), which has also been used in prior reinforcement learning formulations for HPC and cluster scheduling problems [de Freitas Cunha and Chaimowicz, 2023]. The scheduling problem is defined as

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \rho_0, \gamma),$$

where $\mathcal{S}$ is the scheduling state space, $\mathcal{A}$ is the action space, P is the transition dynamics induced by the simulator and workload trace, r is the reward function, ρ_0 is the initial-state distribution, and $\gamma \in [0, 1]$ is the discount factor.

2.1 Scheduling Environment

The environment consists of a finite set of GPU nodes and a sequence of submitted jobs, following the common trace-driven evaluation setting used in cluster scheduling studies [Gu et al., 2019, Xiao et al., 2018, Qiao et al., 2021]. Each node has limited CPU, memory, and GPU capacity. Each job j

is associated with a submit time, estimated runtime, CPU demand, memory demand, GPU demand, and optional GPU-type constraints. Jobs are non-preemptive: once launched, a job occupies its allocated resources until completion.

At any time, jobs are divided into waiting, running, and completed sets. Let Q_t denote the waiting queue at decision epoch t , and let B_t denote the currently available cluster resources. A job can be dispatched only if it has already arrived and can be placed using the current placement backend.

2.2 State and Action Space

At decision epoch t , the scheduler observes a bounded visible queue window of the waiting queue,

$$W_t = \{q_1, \dots, q_K\} \subseteq Q_t,$$

where $q_1, \dots, q_K$ are the first K jobs in queue order. This bounded observation avoids an unbounded action space while preserving the most immediate dispatching choices. In the main experiments, the visible window size is $K = 32$.

The state s_t summarizes three types of information:

$$s_t = (X_t, U_t, b_t),$$

where X_t contains features of feasible dispatching actions, U_t represents the visible queue window, and b_t contains global cluster and queue statistics. This structured state representation follows the general idea of learning scheduling decisions from queue, resource, and job-level features, as explored in prior reinforcement learning schedulers [Mao et al., 2016, 2019]. The exact neural encoding of these components is described in Section 4.

Given the current resource state and placement backend, the feasible job set is

$$F_t = \{j \in W_t \mid \text{Place}(j, B_t) \neq \emptyset\}.$$

The action space is then

$$\mathcal{A}(s_t) = F_t \cup \{\text{pass}\}.$$

If $a_t = j \in F_t$, job j is launched and allocated using the fixed placement backend. If $a_t = \text{pass}$, no job is launched at the current decision epoch. In this formulation, the learned policy optimizes dispatching order only, while low-level node placement is handled by a fixed heuristic. This separation allows the evaluation to focus on dispatching decisions rather than confounding the comparison with different placement strategies.

2.3 Transition Dynamics

After an action is selected, the simulator updates the cluster state by allocating resources to newly launched jobs, releasing resources from completed jobs, and inserting newly arrived jobs into the waiting queue. The simulator then advances to the next event time. The elapsed time between two decision epochs,

$$\Delta t_t,$$

is variable because arrivals and completions occur at irregular times. Thus, the transition is written as

$$s_{t+1} \sim P(\cdot \mid s_t, a_t, \Delta t_t).$$

Although job arrivals and runtimes are replayed from a historical trace, the resulting state trajectory still depends on the scheduler’s previous dispatching decisions. This reflects the long-horizon nature of scheduling, where each decision can affect later queue composition, resource availability, and service balance.

2.4 Objective

The goal is to learn a dispatching policy

$$\pi_\theta(a_t \mid s_t)$$

that improves long-term service fairness while preserving conventional scheduling efficiency. During training, the policy maximizes the expected discounted return

$$J(\pi_\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T-1} \gamma^t r_t \right],$$

where $\tau = (s_0, a_0, s_1, a_1, \dots)$ is generated by interacting with the trace-driven simulator.

For evaluation and checkpoint selection, a penalty-based composite objective is used:

$$\min_{\theta} \quad \text{CFS Normalized Service-Lag AUC}(\pi_\theta) + \lambda_{\text{gate}} \Omega_{\text{gate}}(\pi_\theta),$$

where

$$\Omega_{\text{gate}}(\pi_\theta) = \phi_s + \phi_w + \phi_{95} + \phi_f + \phi_u,$$

and

$$\begin{aligned} \phi_s &= \max \left(0, \frac{\text{AvgSlowdown}(\pi_\theta)}{\alpha_s} - 1 \right), \quad \phi_w = \max \left(0, \frac{\text{AvgWait}(\pi_\theta)}{\alpha_w} - 1 \right), \quad \phi_{95} = \max \left(0, \frac{\text{P95Wait}(\pi_\theta)}{\alpha_{95}} - 1 \right), \\ \phi_f &= \max \left(0, \frac{\alpha_f - \text{Jain}(\pi_\theta)}{\alpha_f} \right), \quad \phi_u = \text{UnfinishedJobs}(\pi_\theta). \end{aligned}$$

The gate limits $\alpha_s, \alpha_w, \alpha_{95}, \alpha_f$ are configurable tolerance thresholds rather than hard feasibility constraints. This design is motivated by constrained reinforcement learning and constrained Markov decision process formulations, where a learned policy should optimize the main objective while avoiding unacceptable violations of auxiliary constraints [Achiam et al., 2017, Altman, 2021]. The criterion reflects the intended trade-off: the scheduler should reduce persistent service-lag imbalance while keeping waiting time, slowdown, tail latency, fairness, and completion reliability within an acceptable envelope.

2.5 Assumptions

The formulation relies on four main assumptions. First, arrivals and runtimes are replayed from historical traces, so the policy does not control the workload sequence. Second, jobs are non-preemptive and cannot be migrated after launch. Third, placement is handled by a fixed backend, so the RL agent learns dispatching decisions rather than node-level allocation. Finally, the policy observes only a bounded top- K queue window, which makes the decision problem tractable but prevents the agent from directly inspecting the entire waiting queue.

3 Empirical Evaluation

To assess the effectiveness of the proposed GLIDE framework for GPU cluster scheduling, a trace-driven experimental protocol is designed to evaluate performance from multiple perspectives, including scheduling efficiency, tail latency, fairness, service-lag imbalance, and learned-policy behavior. Trace-driven evaluation is commonly used in cluster scheduling studies because it allows different scheduling policies to be compared under the same workload sequence and resource setting [Gu et al., 2019, Xiao et al., 2018, Qiao et al., 2021, Mahajan et al., 2020]. All policies are evaluated under the same First-Fit placement backend so that the comparison focuses on dispatching decisions rather than node-placement differences.

3.1 Performance Metrics

The following key performance metrics are adopted to evaluate the proposed method. These metrics correspond to the main result table and the tail/fairness analysis table reported in Table 1 and Table 2. Waiting time and slowdown are standard metrics in parallel job scheduling and cluster scheduling evaluation, while tail metrics are included to reveal starvation-like or extreme-delay behavior that may be hidden by average performance [Feitelson, 2001, Gu et al., 2019, Mahajan et al., 2020].

- **Average Waiting Time:** This metric measures the average duration that jobs remain in the queue before execution. For job i , the waiting time is defined as

$$W_i = T_i^{\text{start}} - T_i^{\text{submit}},$$

where T_i^{submit} and T_i^{start} denote the submit time and start time of the job. Lower values indicate improved scheduling efficiency.

- **95th Percentile Waiting Time:** This metric measures the 95th percentile of job waiting time. It captures tail latency and reflects whether a small fraction of jobs experience long queueing delays.
- **99th Percentile Waiting Time:** This metric further evaluates extreme tail latency by measuring the 99th percentile of waiting time. It is useful for identifying starvation-like behavior that may be hidden by average waiting time.
- **Average Job Slowdown:** Job slowdown normalizes completion delay by runtime and reflects the perceived delay from the user’s perspective. For job i , slowdown is defined as

$$S_i = \frac{W_i + R_i}{\max(R_i, 1)},$$

where R_i denotes job runtime. Lower average slowdown indicates better normalized scheduling efficiency.

- **95th Percentile Job Slowdown:** This metric measures the 95th percentile of slowdown and evaluates whether the scheduler creates high relative delay for tail jobs, especially short jobs that are sensitive to waiting time.
- **99th Percentile Job Slowdown:** This metric measures the 99th percentile of slowdown and captures more extreme relative-delay outliers.
- **Maximum Job Slowdown:** This metric reports the largest slowdown among completed jobs. It is used to evaluate the worst-case relative delay produced by a scheduling policy.
- **Jain’s Fairness Index:** Jain’s fairness index is computed over inverse slowdown values. Let $x_i = 1/S_i$, where S_i is the slowdown of job i . The fairness score is

$$J(x) = \frac{\left(\sum_{i=1}^N x_i\right)^2}{N \sum_{i=1}^N x_i^2}.$$

Higher values indicate more balanced normalized performance across jobs [Jain et al., 1984].

- **CFS Normalized Service-Lag AUC:** This is the primary fairness-oriented metric in the evaluation. It calculates the area under the curve of the maximum normalized positive service lag among all GPU-demand groups over time, i.e., it tracks the worst-off group. A lower value indicates that the scheduler produces less persistent under-service for any single GPU-demand group. This metric is inspired by fair-share scheduling and CFS-style service tracking, where accumulated service imbalance is considered in scheduling decisions [Wong et al., 2008, Mahajan et al., 2020].
- **CFS Maximum Normalized Service Lag:** This metric measures the largest normalized service lag observed during the simulation. While service-lag AUC captures accumulated imbalance, this metric captures the worst instantaneous imbalance.
- **CFS Lag-over-0.5 Ratio:** This metric measures the fraction of service-lag samples whose normalized lag is greater than 0.5. It provides an interpretable threshold-based view of how often the system enters a high-lag state.
- **GPU-Group Slowdown Maximum Gap:** This metric measures the maximum difference in average slowdown among GPU-demand groups. Let $\bar{S}_g$ be the average slowdown of jobs in group g . The metric is defined as

$$\max_{g,g'} |\bar{S}_g - \bar{S}_{g'}|.$$

Lower values indicate more balanced slowdown across job-size groups.

Primary Metric. Among the above metrics, **CFS Normalized Service-Lag AUC** is used as the primary term in checkpoint selection. This choice is motivated by the fact that average waiting time and average slowdown do not explicitly capture persistent group-level under-service. A scheduler may appear efficient on average while repeatedly delaying jobs from a specific GPU-demand group.

CFS Normalized Service-Lag AUC directly measures this type of imbalance, and the final score adds soft penalties for efficiency-gate violations.

Intuitively, this metric captures cases where a specific GPU-demand group is repeatedly underserved even when the overall average waiting time or slowdown appears acceptable. For example, if small-GPU jobs are scheduled quickly while large-GPU jobs repeatedly wait behind them, average metrics may hide this persistent group-level imbalance. CFS Normalized Service-Lag AUC is designed to expose this behavior.

Efficiency and Fairness Trade-off. The evaluation does not rely on a single metric. Instead, GLIDE is expected to reduce CFS Normalized Service-Lag AUC while keeping conventional scheduling metrics such as average waiting time, average slowdown, P95 waiting time, Jain fairness, and unfinished jobs within a configurable tolerance envelope. P99 waiting time, P99 slowdown, and maximum slowdown are also reported as tail diagnostics, so that a degenerate policy cannot improve fairness-related metrics by simply delaying dispatches or reducing throughput. This multi-metric evaluation follows the broader practice in cluster scheduling, where latency, slowdown, fairness, and tail behavior must be considered together [Feitelson, 2001, Gu et al., 2019, Mahajan et al., 2020, Qiao et al., 2021].

3.2 Baseline Methods

The proposed method is compared against representative dispatching policies, including classical queue-ordering heuristics, runtime-aware heuristics, resource-aware heuristics, QoS-aware heuristics, and placement-aware dispatching heuristics. These baselines are included because simple heuristic schedulers often remain strong and interpretable references in GPU and cluster scheduling studies [Gu et al., 2019, Mahajan et al., 2020, Narayanan et al., Qiao et al., 2021].

- **FIFO Scheduling:** Processes jobs strictly in their order of arrival without additional prioritization. This serves as the simplest queue-ordering baseline.
- **Shortest Job First (SJF):** Prioritizes jobs with shorter estimated runtime to reduce average waiting time and average slowdown.
- **Smallest-GPU First:** Prioritizes jobs that request fewer GPUs. This heuristic favors jobs that are easier to place under fragmented resources, but may delay large-GPU jobs.
- **Longest-Wait:** Prioritizes the job with the longest waiting time. This directly addresses starvation risk, but does not consider runtime or placement quality.
- **Slowdown Priority:** Prioritizes jobs according to slowdown-related urgency, favoring jobs whose waiting cost is large relative to runtime.
- **QoS Priority:** Dispatches jobs primarily according to predefined service-class or QoS priority.
- **QoS-SJF:** Combines QoS priority with shortest-job-first ordering. This baseline represents a hybrid priority and runtime-aware heuristic.
- **Dominant Resource:** Ranks jobs according to their dominant resource demand, following the spirit of dominant-resource fairness [Ghods et al., 2011].
- **Best-Fit-Job:** Selects the job that best fits the currently available resources under the fixed placement backend. The name describes the dispatching rule; node placement is still handled by the shared First-Fit backend.
- **Min-Fragmentation:** Selects the job expected to minimize post-dispatch resource fragmentation. The name describes the dispatching rule; node placement is still handled by the shared First-Fit backend.
- **Max-Utilization:** Selects the job that maximizes immediate resource utilization. The name describes the dispatching rule; node placement is still handled by the shared First-Fit backend.

The learned policy is denoted as **GLIDE**. In the main experiment, GLIDE and all heuristic baselines use **First-Fit** placement. Therefore, performance differences should be interpreted as differences in dispatching order rather than differences in low-level node placement.

3.3 Benchmark Tasks or Datasets

The evaluation is conducted using real-world GPU cluster traces.

- **Alibaba Cluster Trace GPU v2023:** This dataset contains job submissions, resource requirements, GPU specifications, and node information from a production GPU cluster. It is used as the main benchmark in the experiments.
- **Evaluation Split:** To avoid train-test leakage, GLIDE is trained on 1000 jobs starting from offset 0 and evaluated on a disjoint 1000-job window starting from offset 2000. Submit times are re-normalized within each window, but the two job windows remain disjoint.

Evaluation Procedure. A trace-driven simulation environment is constructed based on the Alibaba Cluster Trace GPU v2023 trace. At each event-driven decision step, the scheduler selects one job from the bounded visible queue window or chooses the pass action. The selected job is then allocated using a predefined First-Fit placement policy. Jobs are non-preemptive and occupy their allocated resources until completion.

All policies are evaluated using the same evaluation trace, same node subset, same arrival-time scaling, and same First-Fit placement backend. Performance metrics are computed over the full scheduling horizon. For GLIDE, the reported result corresponds to the selected validation checkpoint under the CFS Normalized Service-Lag AUC primary term and the efficiency-gate penalty envelope.

4 Methodology

This project proposes **GLIDE**, a GPU Lag-Informed Decision Engine for learning dispatching decisions in the SMDP defined in Section 2. The key design choice is to separate dispatching from placement. GLIDE learns which feasible job to launch, while a fixed placement backend determines where the selected job is allocated. This design keeps the action space tractable and allows the evaluation to isolate dispatching quality from node-level placement effects. Similar separations between high-level scheduling decisions and lower-level resource-management mechanisms are common in cluster scheduling systems and RL-based resource-management studies [Mao et al., 2016, 2019, Gu et al., 2019].

4.1 Policy Input Construction

At each decision epoch, GLIDE receives the structured state $s_t = (X_t, U_t, b_t)$. The feasible-action matrix X_t contains one row per feasible job and one additional row for the pass action. Job features include waiting time, estimated runtime, requested GPU count, placement-fit statistics, slowdown-related quantities, and service-lag-related quantities.

The queue-set input U_t represents the bounded visible queue window. It allows the policy to compare feasible jobs against nearby waiting jobs instead of scoring each action in isolation. The global vector b_t summarizes queue length, waiting-time and slowdown statistics, utilization, fragmentation, feasible-job ratios, recent debt gaps, node-level free-GPU statistics, and service-lag summaries. This structured representation follows the general idea of encoding queue, job, and resource states for learned scheduling policies [Mao et al., 2016, 2019, de Freitas Cunha and Chaimowicz, 2023].

In the main configuration, the visible window size is 32. Therefore, GLIDE does not rely on scoring the entire waiting queue; after feasibility filtering, the policy deterministically evaluates all feasible jobs within this bounded visible window, plus the pass action.

4.2 Queue-aware Recurrent Policy

A scheduling decision depends on both the current queue composition and recent system evolution. GLIDE therefore combines a queue-set encoder with recurrent memory. The queue-set encoder captures the visible waiting jobs, while the recurrent state summarizes recent event-driven system dynamics.

The global state vector is embedded by an MLP:

$$h_t^b = f_b(b_t).$$

The visible queue rows are embedded and processed by multi-head self-attention:

$$E_t = f_q(U_t), \quad \tilde{E}_t = \text{MHA}(E_t, E_t, E_t).$$

Mean and max pooling are then used to obtain the queue context:

$$h_t^q = f_{\text{pool}} \left(\text{mean}(\tilde{E}_t), \text{max}(\tilde{E}_t) \right).$$

The state embedding and queue context are fused and passed into a GRU:

$$o_t = f_o([h_t^b, h_t^q]), \quad h_t = \text{GRU}(o_t, h_{t-1}).$$

The recurrent hidden state h_t carries information across event-driven decisions, which is useful because scheduling decisions have delayed effects on future queue composition and resource availability.

For each feasible action j , its feature vector is embedded as

$$v_{t,j} = f_x(x_{t,j}).$$

The actor scores each action using the action embedding, recurrent context, interaction features, feasible-action summaries, and queue context:

$$z_{t,j} = f_\theta([v_{t,j}, h_t, v_{t,j} \odot h_t, \bar{v}_t, v_t^{\text{max}}, h_t^q]),$$

where $\bar{v}_t$ and v_t^{max} are the mean and max pooled feasible-action embeddings. The dispatching policy is a categorical distribution:

$$\pi_\theta(a_t = j \mid s_t) = \frac{\exp(z_{t,j}/\tau)}{\sum_{k \in \mathcal{A}(s_t)} \exp(z_{t,k}/\tau)}.$$

During training, actions are sampled from this distribution. During evaluation, GLIDE selects the action with the highest score.

4.3 Service-lag-aware Reward

The reward is designed to reduce persistent service imbalance while discouraging latency collapse. Let $L_g(t)$ be the normalized positive service lag of GPU-demand group g , and define the system-level lag potential as

$$\Phi_t = \max_{g \in G} L_g(t).$$

This lag-based design is inspired by fair-share scheduling and CFS-style service tracking, where accumulated service imbalance is used to guide scheduling behavior [Wong et al., 2008, Mahajan et al., 2020].

For each transition, the log-scaled lag level, growth, and reduction are computed as

$$\ell_t = \log(1 + \Phi_t), \quad \Delta_t^+ = [\ell_{t+1} - \ell_t]_+, \quad \Delta_t^- = [\ell_t - \ell_{t+1}]_+.$$

The step-level shaping reward is

$$r_t^{\text{step}} = -w_{\text{level}}\ell_{t+1} - w_{\text{growth}}\Delta_t^+ + w_{\text{reduce}}\Delta_t^- - w_{\text{queue}}c_{\text{queue}} - w_{\text{pass}}c_{\text{pass}}.$$

The efficiency-gate penalty is not applied at every step. Because average slowdown, average waiting time, P95 waiting time, Jain fairness, and unfinished-job counts are only known after an episode finishes, the gate penalty is attached to the terminal transition:

$$r_t = \begin{cases} r_t^{\text{step}}, & \text{for intermediate decisions,} \\ r_t^{\text{step}} - w_{\text{gate}}c_{\text{gate}}, & \text{for the terminal transition of an episode.} \end{cases}$$

The first three terms make service-lag reduction the main learning signal. The queue and pass penalties discourage excessive queue buildup and artificial idling. The gate penalty is computed from normalized violations of average slowdown, average waiting time, P95 waiting time, Jain fairness, and unfinished-job counts. This penalty-based design follows the broader motivation of constrained reinforcement learning, where policy optimization should account for auxiliary safety or performance constraints in addition to the main reward objective [Achiam et al., 2017, Altman, 2021].

Table 1: Main evaluation results on the Alibaba Cluster Trace GPU v2023 evaluation split. All policies use the same First-Fit placement backend. Lower is better for waiting time, slowdown, and CFS lag AUC; higher is better for Jain fairness. The table includes all policies from the baseline result file.

Policy	Avg. Wait	P95 Wait	Avg. Slowdown	P95 Slowdown	Jain	CFS Lag AUC
FIFO	1138.84	3424.25	4.729	17.398	0.7699	144.154
SJF	932.79	2769.25	3.836	13.188	0.7976	14.274
Smallest-GPU	997.12	3027.00	4.182	13.883	0.7928	30.758
Longest-Wait	1138.84	3424.25	4.729	17.398	0.7699	144.154
Slowdown Priority	927.73	2724.00	3.847	13.353	0.7994	14.073
QoS Priority	1180.75	3443.25	4.802	18.212	0.7624	191.238
QoS-SJF	1035.31	2977.25	4.213	15.377	0.7766	15.234
Dominant Resource	1164.30	3351.50	4.730	17.681	0.7629	166.048
Best-Fit-Job	1173.06	3419.00	4.780	17.909	0.7624	175.674
Min-Fragmentation	1199.69	3464.75	4.855	18.212	0.7609	228.975
Max-Utilization	1164.30	3351.50	4.730	17.681	0.7629	166.048
GLIDE (ours)	905.64	2685.25	3.776	13.069	0.7999	13.742

4.4 Training and Model Selection

GLIDE is trained with PPO through online interaction with the trace-driven simulator. Although the workload is replayed from historical traces, the training process is not offline RL: each update collects new trajectories by rolling out the current policy. PPO is adopted because its clipped surrogate objective is widely used to stabilize policy-gradient optimization [Schulman et al., 2017].

For each trajectory, generalized advantage estimates are computed as

$$\hat{A}_t = \sum_{\ell=0}^{T-t-1} (\gamma\lambda)^\ell \delta_{t+\ell}, \quad \delta_t = r_t + \gamma V_\psi(s_{t+1}) - V_\psi(s_t).$$

Generalized advantage estimation reduces variance while preserving a controllable bias-variance trade-off in policy-gradient learning [Schulman et al., 2015]. The actor is optimized with the clipped PPO loss:

$$\mathcal{L}_{\text{actor}} = -\mathbb{E}_t \left[\min \left(\rho_t(\theta) \hat{A}_t, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right],$$

where

$$\rho_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}.$$

The critic is trained with a value loss, and an entropy bonus encourages exploration.

After each update, the policy is evaluated on a held-out trace window. The checkpoint with the lowest penalty-based composite score is selected, where CFS normalized service-lag AUC is the primary term and the efficiency gates contribute soft penalties. Behavioral diagnostics, including selected risk rank and pass-action ratio, are also reported to check whether the policy improves dispatching order without relying on artificial idling.

5 Experimental Results

5.1 Main Results

Under our controlled trace-driven setting, where all methods share the same First-Fit placement backend, GLIDE achieves the best performance among the evaluated dispatching policies on CFS normalized service-lag AUC, average waiting time, P95 waiting time, average slowdown, P95 slowdown, and Jain fairness. Relative to the strongest heuristic on CFS lag AUC, Slowdown Priority, GLIDE reduces CFS lag AUC by 2.35%. Relative to SJF, GLIDE reduces CFS lag AUC by 3.73%, average slowdown by 1.57%, and average waiting time by 2.91%. These improvements indicate a better dispatching order rather than an advantage from a different placement backend.

Although the numerical gains are moderate, they are consistent across service-lag, average latency, slowdown, and tail-related metrics. This is meaningful because SJF and slowdown-priority are strong heuristic baselines that already optimize conventional scheduling objectives effectively.

Table 2: Tail-latency and fairness-oriented metrics on the same evaluation split. Lower is better for all columns. The table includes all heuristic baselines and GLIDE.

Policy	P99 Wait	P99 Slowdown	Max Slowdown	CFS Max Lag	Lag > 0.5	GPU Group Slow Gap
FIFO	5018.15	57.323	168.19	0.756	0.445	3.448
SJF	3564.55	49.911	137.38	0.694	0.194	2.552
Smallest-GPU	4739.65	54.574	173.61	0.726	0.434	2.756
Longest-Wait	5018.15	57.323	168.19	0.756	0.445	3.448
Slowdown Priority	3540.40	48.736	142.06	0.691	0.193	2.532
QoS Priority	5566.85	60.276	169.56	0.753	0.455	3.607
QoS-SJF	3844.20	51.849	154.81	0.707	0.208	3.096
Dominant Resource	5114.65	60.205	166.50	0.745	0.454	3.678
Best-Fit-Job	4665.25	60.260	168.19	0.753	0.459	3.728
Min-Fragmentation	5566.85	61.397	175.19	0.752	0.454	3.803
Max-Utilization	5114.65	60.205	166.50	0.745	0.454	3.678
GLIDE (ours)	3457.05	47.377	129.50	0.686	0.189	2.367

Table 3: Policy diagnostics for the selected GLIDE checkpoint. Risk rank is computed within the feasible action set from the bounded top-32 visible queue window.

Diagnostic	GLIDE
Highest-risk feasible action selected ratio	81.2%
Mean selected risk rank	1.31
Pass action ratio	0.0
Mean feasible jobs	3.09
Mean action-set size	4.09
Average debt gap	0.0156

5.2 Tail and Fairness Analysis

The tail metrics support the main result. GLIDE obtains the lowest P99 wait, P99 slowdown, maximum slowdown, maximum normalized CFS lag, lag-over-0.5 ratio, and GPU-group slowdown gap. This suggests that the CFS-lag improvement is not a reward-hacking artifact: the selected checkpoint stays within the configured efficiency envelope and also improves tail behavior.

5.3 Policy Diagnostics

The diagnostic results show that GLIDE frequently selects the highest-risk feasible action, not the globally most unfair job in the entire waiting queue. This distinction matters: the policy only chooses among jobs that are visible and feasible under the current First-Fit backend. The pass action is available, but the best checkpoint does not use it, indicating that the improvement is not caused by artificial idling.

5.4 Training Dynamics

The selected checkpoint occurs at update 33. Validation CFS lag AUC drops from 12311.41 at update 0 to 13.74 at update 33. Later checkpoints drift upward, with the final checkpoint performing substantially worse. We therefore report the selected validation checkpoint under the penalty-based composite score rather than the final checkpoint. This is a practical limitation of PPO in this setting and motivates validation-based early stopping.

6 Expected Contributions of Each Team Member

To ensure effective collaboration, the project is divided into several key components, including environment simulation, model design, experimentation, and analysis. Each member is responsible for specific aspects of the project while also contributing to joint development, discussion, and integration.

- **Haopei Teng:** Responsible for developing the reinforcement learning models, including policy architecture design, reward function implementation, and training pipeline construc-

tion. She will also explore different learning algorithms and improve model stability and performance.

- **Juiyun Chang:** Responsible for designing and implementing the simulation environment based on real-world GPU cluster traces, including data preprocessing, event-driven simulation, and system state representation. He will also assist in integrating the scheduling policy into the simulation framework.
- **Jinjia Zheng:** Responsible for experimental design and evaluation, including implementing baseline methods, defining evaluation metrics, conducting experiments, and analyzing results. He will also focus on interpreting the trade-offs between scheduling efficiency and fairness.

All members will collaboratively participate in idea development, model refinement, and report writing. Regular discussions and progress reviews will be conducted to ensure consistency and alignment across different components. The team will also jointly contribute to debugging, performance optimization, and final presentation preparation.

As the project progresses, task allocation may be adjusted based on practical needs and individual strengths while maintaining a balanced contribution among all members.

References

- Juncheng Gu, Mosharaf Chowdhury, Kang G Shin, Yibo Zhu, Myeongjae Jeon, Junjie Qian, Hongqiang Liu, and Chuanxiong Guo. Tiresias: A {GPU} cluster manager for distributed deep learning. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*, pages 485–500, 2019.
- Wencong Xiao, Romil Bhardwaj, Ramachandran Ramjee, Muthian Sivathanu, Nipun Kwatra, Zhenhua Han, Pratyush Patel, Xuan Peng, Hanyu Zhao, Quanlu Zhang, et al. Gandiva: Introspective cluster scheduling for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pages 595–610, 2018.
- Aurick Qiao, Sang Keun Choe, Suhas Jayaram Subramanya, Willie Neiswanger, Qirong Ho, Hao Zhang, Gregory R Ganger, and Eric P Xing. Pollux: Co-adaptive cluster scheduling for goodput-optimized deep learning. In *15th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 21)*, 2021.
- Kshiteej Mahajan, Arjun Balasubramanian, Arjun Singhvi, Shivaram Venkataraman, Aditya Akella, Amar Phanishayee, and Shuchi Chawla. Themis: Fair and efficient {GPU} cluster scheduling. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 289–304, 2020.
- Chee Siang Wong, Ian Tan, Rosalind Deena Kumari, and Fun Wey. Towards achieving fairness in the linux scheduler. *ACM SIGOPS Operating Systems Review*, 42(5):34–43, 2008.
- Joshua Achiam, David Held, Aviv Tamar, and Pieter Abbeel. Constrained policy optimization. In *International conference on machine learning*, pages 22–31. Pmlr, 2017.
- Eitan Altman. *Constrained Markov decision processes*. Routledge, 2021.
- Rajendra K Jain, Dah-Ming W Chiu, William R Hawe, et al. A quantitative measure of fairness and discrimination. *Eastern Research Laboratory, Digital Equipment Corporation, Hudson, MA*, 21(1):2022–2023, 1984.
- Hongzi Mao, Mohammad Alizadeh, Ishai Menache, and Srikanth Kandula. Resource management with deep reinforcement learning. In *Proceedings of the 15th ACM workshop on hot topics in networks*, pages 50–56, 2016.
- Hongzi Mao, Malte Schwarzkopf, Shaileshh Bojja Venkatakrisnan, Zili Meng, and Mohammad Alizadeh. Learning scheduling algorithms for data processing clusters. In *Proceedings of the ACM special interest group on data communication*, pages 270–288. 2019.
- Renato Luiz de Freitas Cunha and Luiz Chaimowicz. An smdp approach for reinforcement learning in hpc cluster schedulers. *Future Generation Computer Systems*, 139:239–252, 2023.
- Yifan Yang, Gang Chen, Hui Ma, Cong Zhang, Zhiguang Cao, and Mengjie Zhang. Graph assisted offline-online deep reinforcement learning for dynamic workflow scheduling. In *The Thirteenth International Conference on Learning Representations*, 2025.
- Jin Yang, Liang Bao, Wenjing Liu, Rong Yang, and Chase Q Wu. On a meta learning-based scheduler for deep learning clusters. *IEEE Transactions on Cloud Computing*, 11(4):3631–3642, 2023.
- Dror G Feitelson. Metrics for parallel job scheduling and their convergence. In *Workshop on Job Scheduling Strategies for Parallel Processing*, pages 188–205. Springer, 2001.
- D Narayanan, K Santhanam, F Kazhamiaka, et al. Gavel: Heterogeneity-aware cluster scheduling policies for deep learning workloads. In *Proceedings of USENIX OSDI*, volume 20, pages 481–497.
- Junyeol Ryu and Jeongyoon Eo. Network contention-aware cluster scheduling with reinforcement learning. In *2023 IEEE 29th International Conference on Parallel and Distributed Systems (ICPADS)*, pages 2742–2745. IEEE, 2023.
- Ali Ghodsi, Matei Zaharia, Benjamin Hindman, Andy Konwinski, Scott Shenker, and Ion Stoica. Dominant resource fairness: Fair allocation of multiple resource types. In *8th USENIX symposium on networked systems design and implementation (NSDI 11)*, 2011.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*, 2015.